Smart Slot Allocation System Using IoT

This document serves as the Technical Reference Manual for the IoT Smart Slot Allocation System, providing comprehensive details on its definition, architectural components, operational principles, and physical implementation.

1. Project Definition & System Scope

The Smart Slot Allocation System is an Internet of Things (IoT) based solution engineered to modernize vehicle parking management. It directly addresses the inefficiencies of manual parking systems, which contribute to urban traffic congestion and increased carbon emissions.

This architecture allows for the automated monitoring of parking vacancy and the remote booking of spaces, optimizing the allocation of parking assets without the requirement for on-site personnel.

2. Key IoT Components

The system is built upon a modular architecture of sensing, processing, and visualization components.

ESP32 Dev Kit-C

Infrared (IR) / Ultrasonic Sensors:

LCD Display (16x2)

LED Indicators

2.1 ESP32 Microcontroller (Central Unit)

The ESP32 acts as the core processor of the system, handling analog-to-digital conversion, executing the operational logic, and managing connection fo cloud synchronization [cite: 1146-1148].

2.2 Infrared (IR) Obstacle Sensors

Used for primary object detection. The sensor consists of an IR transmitter and receiver. Vehicle presence is detected when the transmitted light is reflected back to the receiver, pulling the corresponding GPIO pin low [cite: 1137-1139].

2.3 16x2 LCD Display (Local HMI)

The local Human-Machine Interface (HMI) provides instant feedback to arriving drivers. It is updated in real-time to show the availability status of specific slots (e.g., EMPTY, BOOKED, FULL) [cite: 1158-1161].

2.4 LED State Indicators

A redundant visual system utilizing Red and Green LEDs. The Green LED indicates vacancy, while the Red LED signifies that a slot is either physically occupied or remotely reserved [cite: 951-952].

3. Working Principle & Operational Logic

The hardware employs a recursive logic loop known as the 'Sense-Resolve-Synchronize' cycle, operating at high frequency to ensure minimal data latency.

3.1 The Resolve-Conflict Logic

The microcontroller must manage conflicts between physical sensing and remote database flags (e.g., if a user seeing parking slot are “Empty” or “Full”). The system implements a hierarchical priority model [cite: 1129-1132]:

- **Physical Presence (FULL):** Has the highest priority. If the sensor detects a car, the database must reflect 'FULL', regardless of booking state.
- **Vacancy (EMPTY):** Default state if neither prior condition is met.
- If a slot is physically detected as full, the status is set to **"FULL"**.
- If no car is detected, the status defaults to **"EMPTY"**

4. System Flow & Connectivity (IoT only)

The connection flow diagram illustrates the movement of data between the physical hardware layer and the cloud-based database.

[START] → (Initialize Peripherals & Wi-Fi Stack)

↓

[LOOP BEGIN] → (Sample IR Sensors & Fetch Server Flags)

↓

(Execute Resolve-Conflict Logic & Update Local LCD/LEDs)

↓

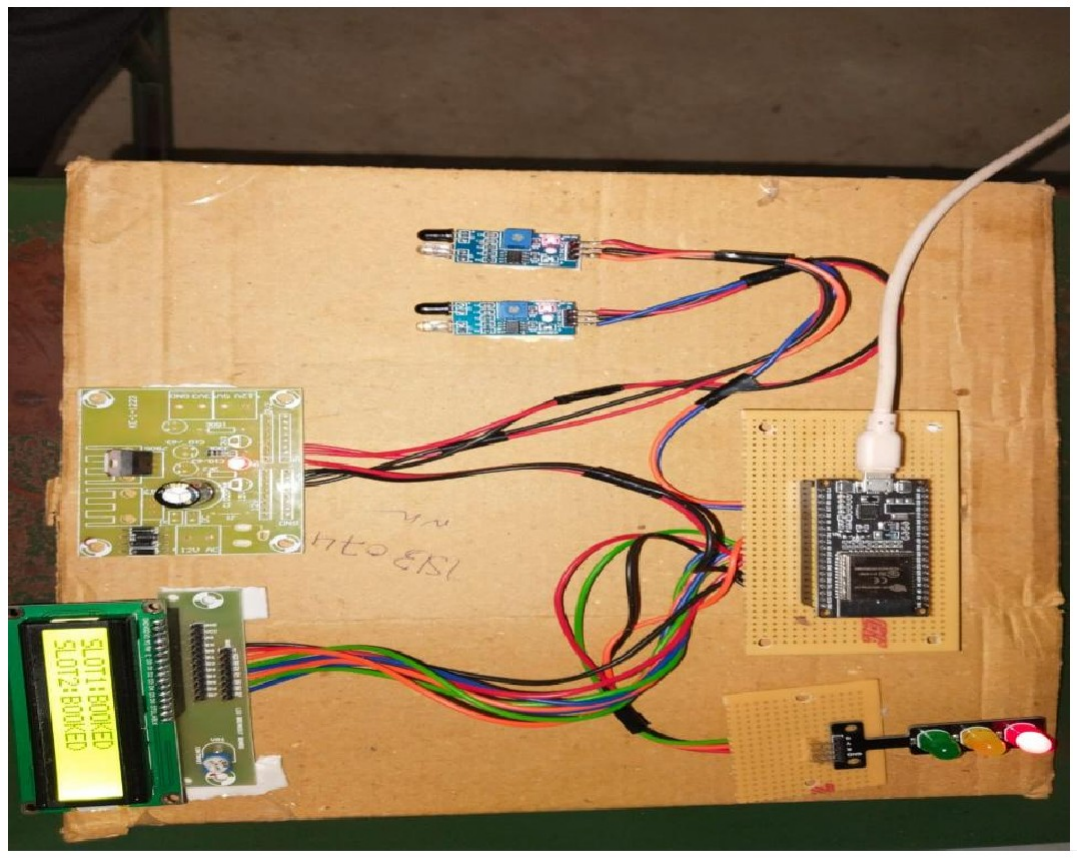
(Publish Final Validated State via HTTP POST)

↓

(Wait defined interval) → [LOOP BACK]

5.Setup Instructions

- **Wiring:** Connect sensors, LCD, and LEDs to the ESP32 according to the Pin Mapping above.
- **Wi-Fi Config:** Update the ssid and pass in the source code to match your local network .
- **Upload:** Use the Arduino IDE to upload the code to the ESP32 Dev Kit-C.
- **Verification:** Monitor the Serial Console at 9600 baud to confirm the "WiFi connected" message and IP address .

FINAL OUTPUT IMAGE

The following image shows the full integration of the system, illustrating the final